AXI4-LITE 실습

- 김범준

TABLE OF CONTENTS

1. Project Summary

2. System Architecture

3. Key Design

4. Result

5. Summary

1. PROJECT SUMMARY

→ **AXI4-Lite Design**

▶ **Tech Stack**

- **Language** : SystemVerilog
- **Tools** : Vivado 2025.2, Visual Studio Code

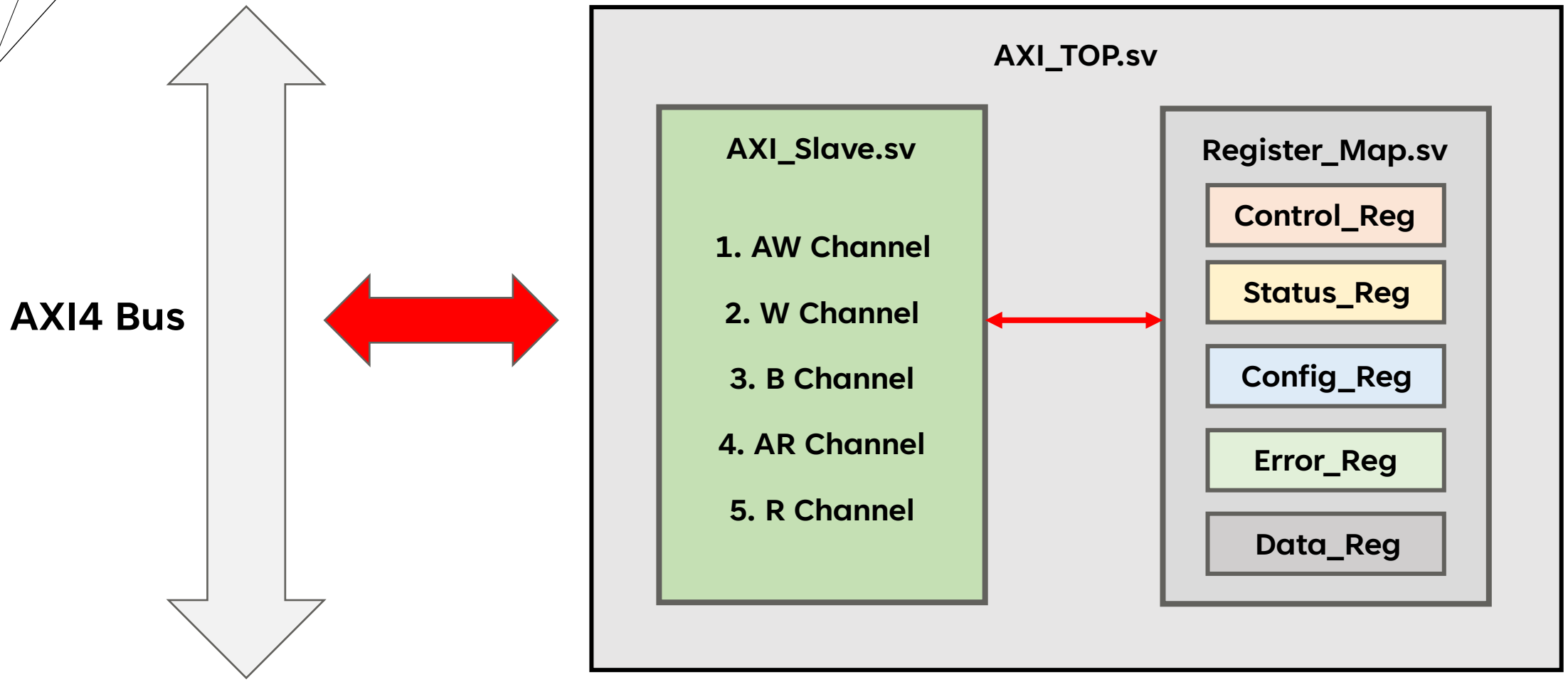
▶ **Duration**

- 2026.02.23 – 2026.03.08

▶ **Project Links**

- **Blog** : bjunverse_ (범이) / 작성글 – velog
- **Github** : bjunverse26/AXI4_Lite: AXI4 Practice1

2. SYSTEM ARCHITECTURE

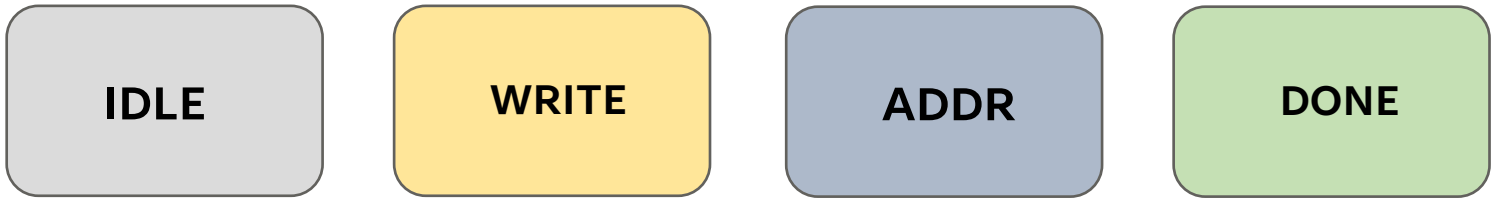


AXI4 Slave Interface

3. KEY DESIGN

▶ Write Transaction

4-State



1. IDLE

- **AW Handshake** : IDLE → WRITE
- **W Handshake** : IDLE → ADDR
- **AW & W Handshake** : IDLE → DONE

2. WRITE

- **W Handshake** : WRITE → DONE

3. ADDR

- **AW Handshake** : ADDR → DONE

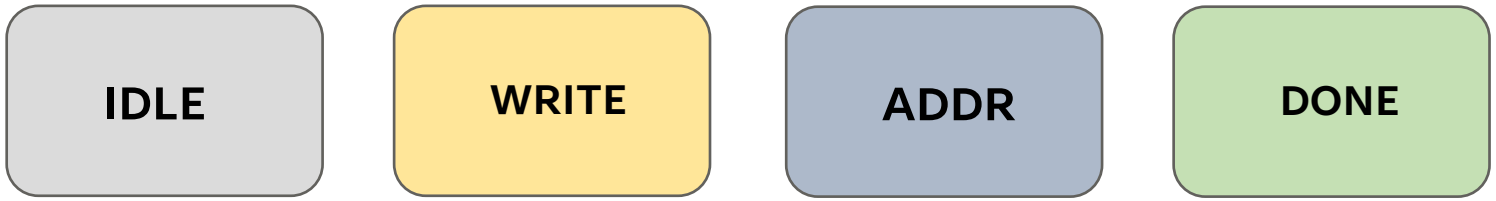
4. DONE

- **B Handshake** : DONE → IDLE

3. KEY DESIGN

▶ Write Transaction

4-State




```
case (current_wstate)
    W_IDLE: begin
        reg_wen <= {NUM_REGS{1'b0}};
        s_axi_awready <= 1'b1;
        s_axi_wready <= 1'b1;

        if (aw_handshake && !w_handshake) begin
            s_axi_awready <= 1'b0;
            waddr_latched <= s_axi_awaddr[9:2];
            current_wstate <= W_WRITE;
        end else if (w_handshake && !aw_handshake) begin
            s_axi_wready <= 1'b0;
            wdata_latched <= s_axi_wdata;
            wstrb_latched <= s_axi_wstrb;
            current_wstate <= W_ADDR;
        end else if (aw_handshake && w_handshake) begin
            s_axi_awready <= 1'b0;
            s_axi_wready <= 1'b0;
            if (s_axi_awaddr[9:2] < NUM_REGS) begin
                reg_wen[s_axi_awaddr[9:2]] <= 1'b1;
                reg_wdata <= s_axi_wdata;
                reg_wstrb <= s_axi_wstrb;
            end
            current_wstate <= W_DONE;
            s_axi_bresp <= 2'b00;
            s_axi_bvalid <= 1'b1;
        end
    end
```

AXI_Slave.sv

```
// Case 1: ADDR -> WRITE
W_WRITE: begin
    if (w_handshake) begin
        s_axi_wready <= 1'b0;
        reg_wdata <= s_axi_wdata;
        reg_wstrb <= s_axi_wstrb;
        current_wstate <= W_DONE;
        s_axi_bresp <= 2'b00;
        s_axi_bvalid <= 1'b1;
        if (waddr_latched < NUM_REGS) reg_wen[waddr_latched] <= 1'b1;
    end
end

// Case 2: WRITE -> ADDR
W_ADDR: begin
    if (aw_handshake) begin
        s_axi_awready <= 1'b0;
        reg_wdata <= wdata_latched;
        reg_wstrb <= wstrb_latched;
        current_wstate <= W_DONE;
        s_axi_bresp <= 2'b00;
        s_axi_bvalid <= 1'b1;
        if (s_axi_awaddr[9:2] < NUM_REGS) reg_wen[s_axi_awaddr[9:2]] <= 1'b1;
    end
end
```

AXI_Slave.sv

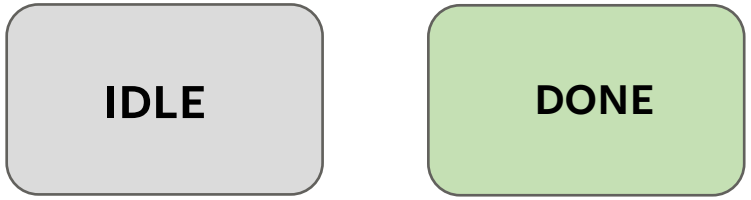
```
        W_DONE: begin
            reg_wen <= {NUM_REGS{1'b0}};

            if (b_handshake) begin
                s_axi_bvalid <= 1'b0;
                current_wstate <= W_IDLE;
            end
        end

    endcase
end
```

AXI_Slave.sv

3. KEY DESIGN

▶ Read Transaction

2-State



1. IDLE
   - **AR Handshake** : IDLE → DONE
2. DONE
   - **R Handshake** : DONE → IDLE

```
case (current_rstate)
    R_IDLE: begin
        s_axi_arready <= 1'b1;

        if (ar_handshake) begin
            s_axi_arready <= 1'b0;
            current_rstate <= R_DONE;
            s_axi_rvalid <= 1'b1;
            s_axi_rresp <= 2'b00;
            if (s_axi_araddr[9:2] < NUM_REGS) begin
                s_axi_rdata <= reg_rdata[s_axi_araddr[9:2]];
            end else begin
                s_axi_rdata <= 32'hDEADBEEF;
            end
        end
    end

    R_DONE: begin
        if (r_handshake) begin
            current_rstate <= R_IDLE;
            s_axi_rvalid <= 1'b0;
        end
    end

endcase
```

AXI_Slave.sv

3. KEY DESIGN

▶ Register Map

- **Control Register** : Write & Read OK
- **Status Register** : Only Read
- **Config Register** : Write & Read OK
- **Error Register** : Only Read
- **Data Register [11:0]** : Write & Read OK

Write Operation : Sequential Logic

Read Operation : Combinational Logic

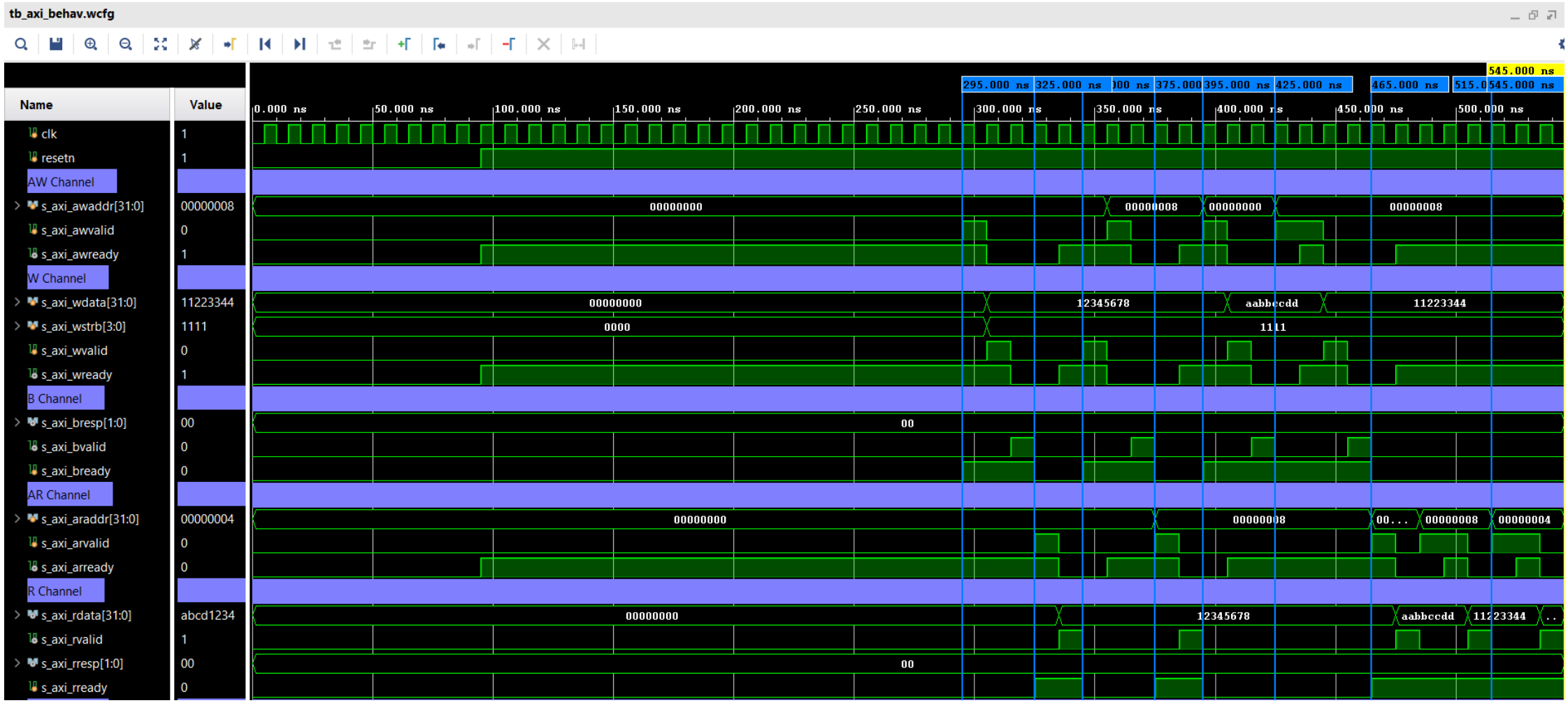
```
// WRITE
always_ff @(posedge clk or negedge resetn) begin
    if (!resetn) begin
        control_reg <= 32'h0;
        config_reg <= 32'h0;
        for (i = 0; i < 12; i++) begin
            data_reg[i] <= 32'h0;
        end
    end else begin
        if (reg_wen[0]) begin
            for (int i = 0; i < 4; i++) begin
                if (reg_wstrb[i]) control_reg[8*i +: 8] <= reg_wdata[8*i +: 8];
            end
        end

        if (reg_wen[2]) begin
            for (int i = 0; i < 4; i++) begin
                if (reg_wstrb[i]) config_reg[8*i +: 8] <= reg_wdata[8*i +: 8];
            end
        end

        for (i = 0; i < 12; i++) begin
            if (reg_wen[i+4]) begin
                for (int j = 0; j < 4; j++) begin
                    if (reg_wstrb[j]) data_reg[i][8*j +: 8] <= reg_wdata[8*j +: 8];
                end
            end
        end
    end
end
```

```
// READ
always_comb begin
    reg_rdata[0] = control_reg;
    reg_rdata[1] = status_reg;
    reg_rdata[2] = config_reg;
    reg_rdata[3] = error_reg;

    for (i = 0; i < 12; i++) begin
        reg_rdata[i+4] = data_reg[i];
    end
end
```

4. RESULT

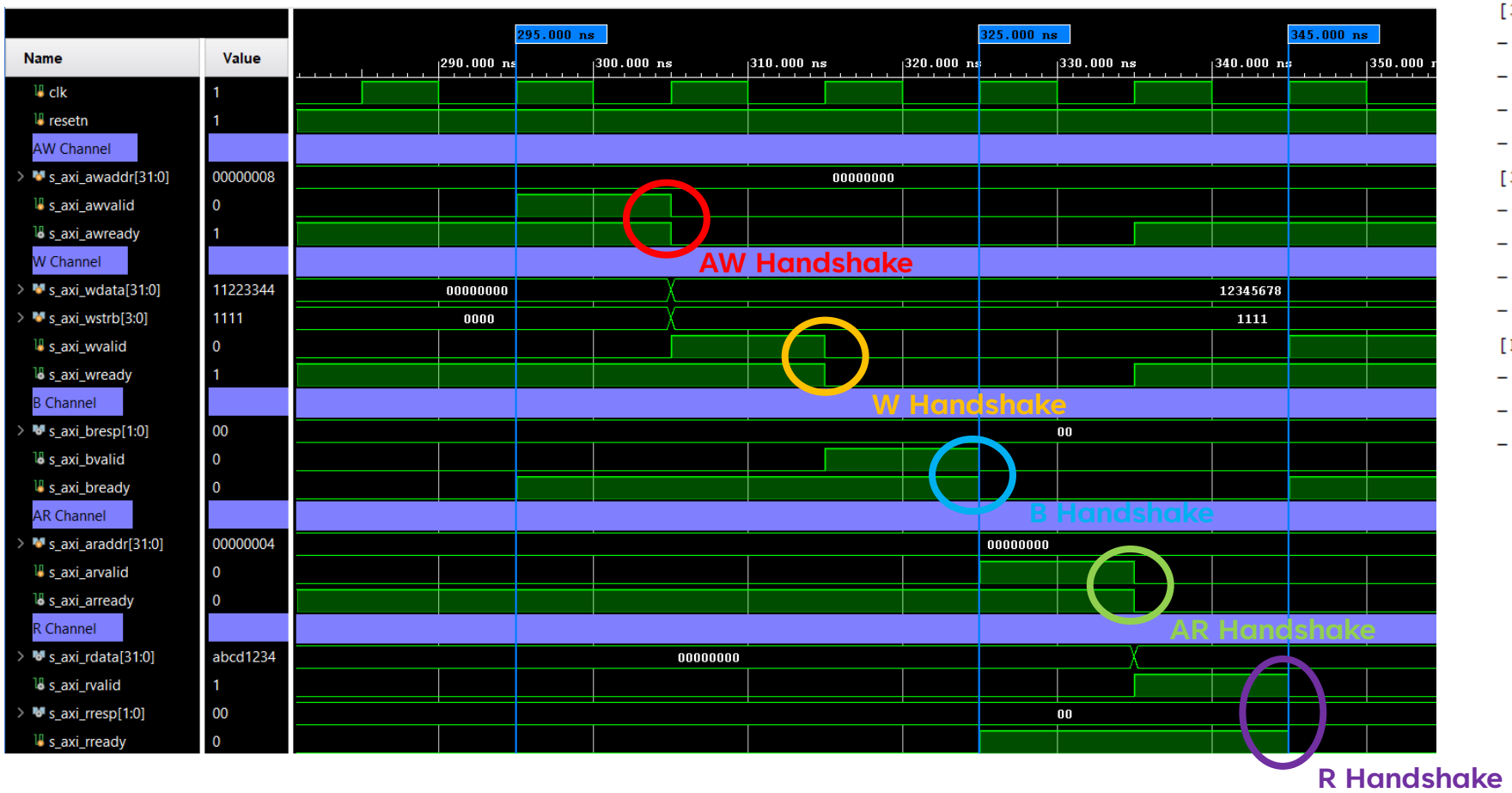
전체 시뮬레이션 파형 – 5 가지 Test 수행

Clock Frequency : 100MHz



4. RESULT

Test 1 : Single Write / Read Test (Case 1 : AW → W)



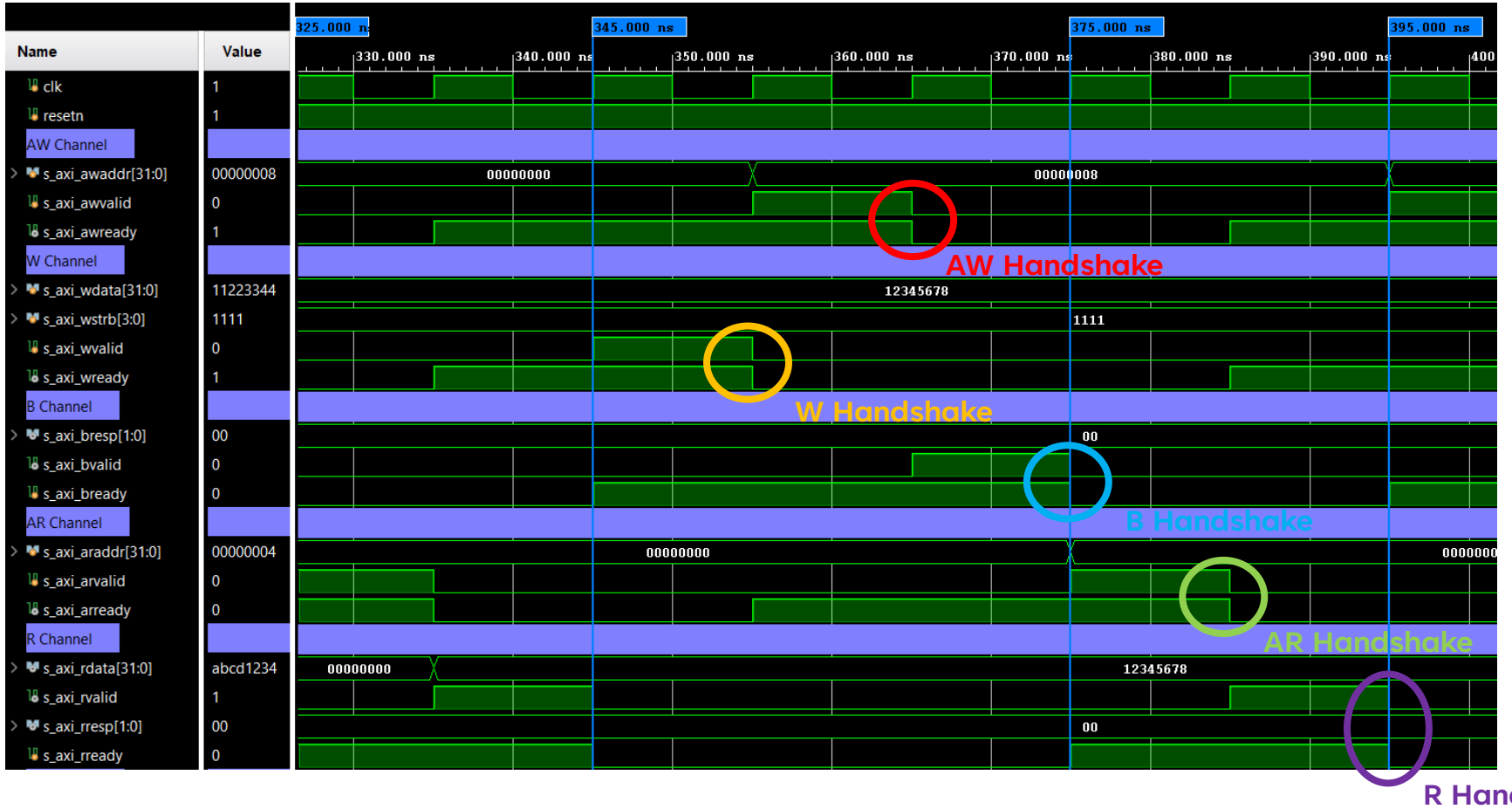

```
========================================
Test 1: Single Write/Read
========================================

[325000 ns] Write1 Transaction
- Address  : 0x00000000
- Data     : 0x12345678
- Response : OKAY
- Duration : 30 ns
[345000 ns] Read Transaction
- Address  : 0x00000000
- Data     : 0x12345678
- Response : OKAY
- Duration : 20 ns
[PASS] Test Case Single Write/Read 1
- Expected : 0x12345678
- Actual   : 0x12345678
- Result   : MATCH
```

1. Write Transaction : **30 ns**

2. Read Transaction : **20 ns**

4. RESULT

Test 2 : Single Write / Read Test (Case 2 : W → AW)



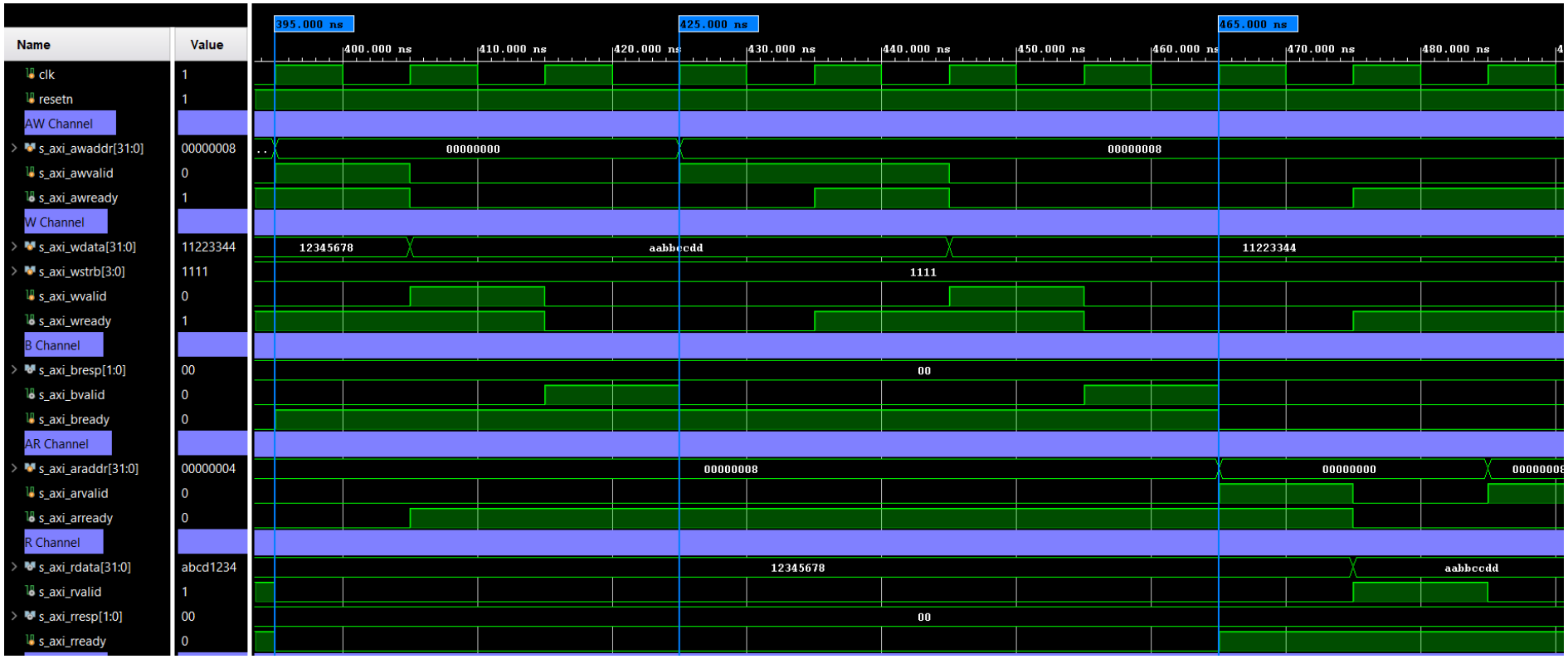

```
========================================
Test 2: Single Write/Read
========================================

[375000 ns] Write2 Transaction
- Address  : 0x00000008
- Data     : 0x12345678
- Response : OKAY
- Duration : 30 ns
[395000 ns] Read Transaction
- Address  : 0x00000008
- Data     : 0x12345678
- Response : OKAY
- Duration : 20 ns
[PASS] Test Case Single Write/Read 2
- Expected : 0x12345678
- Actual   : 0x12345678
- Result   : MATCH
```

1. Write Transaction : **30 ns**

2. Read Transaction : **20 ns**

4. RESULT

Test 3 - 4 : Multiple Write / Read Test (Case 1 : AW → W)



Write 연속 두 번 수행할 때에는 **첫 번째 Write** 는 30ns, **두 번째 Write** 는 40ns가 걸리는 것을 확인할 수 있다.

이유 : DONE State 에서 Handshake 후 **1 cycle Delay 발생**을 원인으로 생각할 수 있다.

4. RESULT

Test 3 - 4 : Multiple Write / Read Test (Case 1 : AW → W)

```
========================================
Test 3: Multiple Sequential Writes
========================================

[425000 ns] Write1 Transaction
- Address  : 0x00000000
- Data     : 0xaabbccdd
- Response : OKAY
- Duration : 30 ns
[465000 ns] Write1 Transaction
- Address  : 0x00000008
- Data     : 0x11223344
- Response : OKAY
- Duration : 40 ns
[485000 ns] Read Transaction
- Address  : 0x00000000
- Data     : 0xaabbccdd
- Response : OKAY
- Duration : 20 ns
[PASS] Test Case Control Register Check
- Expected : 0xaabbccdd
- Actual   : 0xaabbccdd
- Result   : MATCH

[515000 ns] Read Transaction
- Address  : 0x00000008
- Data     : 0x11223344
- Response : OKAY
- Duration : 30 ns
[PASS] Test Case Config Register Check
- Expected : 0x11223344
- Actual   : 0x11223344
- Result   : MATCH
```

Read 연속 두 번 수행할 때에는 **첫 번째 Read** 는 20ns, **두 번째 Read** 는 30ns가 걸리는 것을 확인할 수 있다.

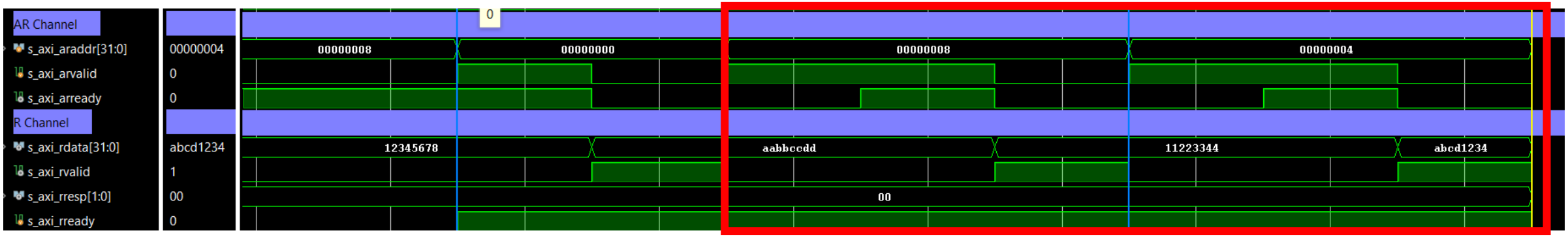
이유 : 마찬가지로 DONE State 에서 Handshake 후 **1 cycle Delay 발생**을 원인으로 생각할 수 있다.

4. RESULT

Test 5 : Read-Only Register Test

Config Register Test

```
=========================================
Test 5: Read-Only Status Register
=========================================

[545000 ns] Read Transaction
- Address  : 0x00000004
- Data     : 0xabcd1234
- Response : OKAY
- Duration : 30 ns
[PASS] Test Case Status Register
- Expected : 0xabcd1234
- Actual   : 0xabcd1234
- Result   : MATCH
```



5. SUMMARY

1. **Write Transaction** 단계에서 Register Map으로 전달되어야 하는 신호는 AW 와 W 가 모두 Handshake 될 때까지 latch 되고 있어야 한다.

2. **Write Transaction** 단계에서 AXI_Slave.sv 쪽에서 wstrb 까지 처리한 후 데이터를 Write 하게 되면 Register Map 이 기존에 갖고 있던 데이터, 즉 wstrb[i] = 0 인 부분까지 지워질 수 있으므로 반드시 Register Map 까지 wstrb을 넘겨야 한다.

3. **Read Transaction** 단계에서 DONE 상태일 때 데이터를 읽는 것으로 처리하면, AR 이 Handshake 발생한 이후부터 R Handshake 발생 이전까지 Read Data가 계속 갱신되므로 반드시 IDLE 상태에서 데이터를 Read 해야 한다.

감사합니다.

김범준

bjunverse26@gmail.com